

App-Prototyping mit Claude: Leitfaden für Produktmanager

Version 1.1 | 07.07.2026

Voraussetzung: Die Einrichtungsanleitung (separates Dokument) wurde einmalig durchgeführt. Claude ist installiert, Vercel und ggf. Supabase sind verbunden.

Inhaltsverzeichnis

- Was dieser Leitfaden tut
 - Übersicht: Die 5 Phasen
 - Schritt 0: Projektordner einrichten
 - Wie Claude den Projektstand speichert
 - Wie du Referenzmaterialien einbringst
 - Phase 1: Spezifikation
 - Phase 2: Initialisierung
 - Phase 3: Konstruktion
 - Phase 4: Validierung
 - Phase 5: Deployment
 - Nach dem Deployment: App weiterentwickeln
 - Anhang A: Fehlereskalationspfad
 - Anhang B: Trigger-Phrasen und Validierungs-Checkliste
 - Anhang C: Kurzglossar
-

Was dieser Leitfaden tut

Dieser Leitfaden führt dich von einer App-Idee bis zu einer öffentlich erreichbaren URL, ohne Programmierkenntnisse. Du beschreibst, was die App tun soll. Claude baut es. Du prüfst, ob es funktioniert. Am Ende hast du einen Prototyp, den du echten Nutzern oder Stakeholdern zeigen kannst.

Was du brauchst:

- Claude Desktop App (installiert und eingerichtet, siehe Einrichtungsanleitung)
- Eine App-Idee mit einer konkreten Funktion

Der Leitfaden führt dich Schritt für Schritt durch fünf Phasen mit Gates (Kontrollpunkten) dazwischen.

Was am Ende herauskommt: Eine öffentlich erreichbare Web-App-URL: navigierbar, funktionsfähig, live.

Übersicht: Die 5 Phasen

Schritt	Was passiert	Wer handelt
Schritt 0	Session-Setup: Projektordner einrichten	Du
Phase 1	Spezifikation: App-Idee beschreiben	Du
Gate 1	Spec vollständig?	Du prüfst
Phase 2	Initialisierung: Projekt einrichten	Claude
Gate 2	App im Browser erreichbar?	Du prüfst
Phase 3	Konstruktion: Features bauen	Claude
Gate 3	Alle Spec-Features implementiert?	Du prüfst
Gate 3b (<i>optional</i>)	Erweiterungsrunde: zusätzliche Features	Claude
Phase 4	Validierung: vollständiger Browser-Test	Du
Gate 4	Alle Akzeptanzkriterien erfüllt?	Du prüfst
Phase 5	Deployment: App auf Vercel deployen	Claude
Gate 5	Öffentliche URL erreichbar?	Du prüfst

Wichtig: Gehe erst zur nächsten Phase, wenn das Gate erfüllt ist.

Schritt 0: Projektordner einrichten

Für jeden neuen Prototyp.

Schritt 0.1: Erstelle einen leeren Ordner auf deinem Computer. Das ist der Projektordner für diesen Prototyp. Wähle einen Namen ohne Leerzeichen, z.B. `mein-prototyp`.

Schritt 0.2: Lade `FRAMEWORK.md` herunter ([GitHub-Link: https://github.com/jusicken/pm-app-prototyping-mit-claude](https://github.com/jusicken/pm-app-prototyping-mit-claude)). Lege die Datei in deinen Projektordner.

Schritt 0.3: Öffne Claude. Klicke auf „**Ordner öffnen**“ und wähle deinen Projektordner aus. Falls Claude aus einem anderen Ordner gestartet wurde: Wähle oben den korrekten Projektordner aus. Claude zeigt immer an, welcher Ordner gerade aktiv ist.

Hinweis: Anfangs wird Claude nach Erlaubnis für bestimmte Aktionen fragen (z.B. Dateien lesen, Befehle ausführen). Klicke immer auf „Erlauben“.

Schritt 0.4: Gib Claude die `FRAMEWORK.md`. Zwei Möglichkeiten:

- **Option A (Empfohlen):** Ziehe die Datei `FRAMEWORK.md` aus dem Projektordner direkt in den Chat-Bereich und sende sie. Kein zusätzlicher Text ist nötig. Claude liest das Dokument und beginnt automatisch mit Phase 1.
- **Option B:** Schreibe in das Chat-Eingabefeld:

```
Lies FRAMEWORK.md
```

und sende die Nachricht ab. (Voraussetzung: `FRAMEWORK.md` liegt bereits in deinem Projektordner; Schritt 0.2 muss abgeschlossen sein.)

Claude liest das Dokument und führt dich direkt zu Phase 1. Du musst nichts weiter eingeben.

Für jeden neuen Prototypen: wiederhole Schritt 0 mit einem neuen, leeren Ordner und einer frischen Kopie von `FRAMEWORK.md`.

Wie Claude den Projektstand speichert

Dieser Abschnitt ist nur zur Information, du musst das jetzt nicht umsetzen.

Nach jedem bestätigten Gate speichert Claude den aktuellen Projektstand automatisch. Du musst nichts kopieren, nichts aufschreiben und nichts außerhalb von Claude festhalten.

Claude verwaltet außerdem `FRAMEWORK.md`, eine kompakte Version des Prozessmodells. Diese Datei liest Claude am Anfang jeder neuen Session, sodass es weiß, in welcher Phase du bist und wie es dich weiterführen soll.

Was das für dich bedeutet:

- Wenn du eine Session schließt und später weitermachst: sage einfach *"Lies `_workspace/FRAMEWORK.md` und `_workspace/project_state.md`."* Claude bringt sich dann selbst auf den aktuellen Stand und sagt dir, was als Nächstes kommt.
- Wenn etwas schiefgeht und du eine neue Session brauchst: dieselbe Anweisung.
- Du musst nie etwas manuell sichern oder einfügen.

Zwischenstand speichern (Session pausieren ohne Gate abzuschließen):

Wenn du eine Session unterbrechen willst, ohne das aktuelle Gate abgeschlossen zu haben, sage zu Claude:

Speichere den aktuellen Zwischenstand. Ich mache eine Pause.

Claude antwortet automatisch — du musst nach dem Senden nichts tun. Claude:

1. Speichert deinen Fortschritt in `_workspace/project_state.md`, auch ohne Gate-Abschluss
2. Fragt dich, ob der laufende Entwicklungsserver gestoppt werden soll: *„Der Server läuft noch auf Port [X]. Soll ich ihn stoppen? Beim nächsten Start wird er automatisch neu gestartet.“*
Du kannst hier mit *"Ja, stoppen"* antworten.
Der Server ist nur eine Vorschau-Umgebung. Dein Code ist in Dateien gespeichert und geht beim Stoppen nicht verloren.
3. Bestätigt dir, dass alles gespeichert ist

Beim nächsten Start verwendest du ST-0 (die Session-Start-Phrase, siehe Anhang B: Gate-Trigger-Phrasen). Claude liest den Zwischenstand und führt dich weiter.

Wie du Referenzmaterialien einbringst

Dieser Abschnitt ist nur zur Information, du musst das jetzt nicht umsetzen.

Du kannst Claude Bilder, Dokumente und Dateien mitgeben: als Kontext für die App-Idee, als Design-Vorlage für ein Feature, oder als Daten-Beispiel.

Einmalige visuelle Referenzen (Mockup, Skizze, Screenshot einer Inspiration, Kompetitor-App):

- Ziehe das Bild direkt in den Chat-Bereich von Claude und sende es mit deinem Prompt.
- Claude kann Bilder lesen. Sage einfach: *"So ungefähr soll es aussehen."*

Persistente Dokumente (PRD, Excel-Tabelle, Nutzerforschungs-Notizen, Brand-Guide):

- Lege die Datei in den `_workspace/inputs/` -Ordner deines Projekts (öffne den Projektordner auf deinem Computer und lege die Datei dort ab).
- Sage Claude dann:

```
Schau dir die Datei [Dateiname] in _workspace/inputs/ an und beziehe sie ein.
```

- Diese Dateien bleiben dauerhaft verfügbar, auch nach Session-Neustarts.

Hinweis: Der `_workspace/` -Ordner wird von Claude automatisch in Phase 2 angelegt — bis dahin existiert er noch nicht in deinem Projektordner.

(Tipp: Auch Bilder können in `_workspace/inputs/` abgelegt werden, wenn du sie als Design-Referenz über mehrere Features hinweg verwenden willst. Sage Claude dann den Dateinamen, statt das Bild jedes Mal neu in den Chat zu ziehen.)

Beide Wege sind optional. Du kannst das Framework ohne jegliche Referenzmaterialien nutzen.

Phase 1: Spezifikation

Ziel: Schriftliches Spec-Dokument mit einer User Story, Akzeptanzkriterien, Datenbankentscheidung und visueller Präferenz.

Schritt 1: Öffne Claude (Schritt 0 muss bereits abgeschlossen sein). Beschreibe Claude in natürlicher Sprache, was du bauen möchtest: was die App tun soll, wer sie nutzen wird, was das Ziel ist. Kein Template nötig.

Optionales Referenzmaterial: Hast du eine Skizze, einen Screenshot oder ein Bild einer App, die als Vorlage dient? Ziehe es in den Chat. Claude bezieht es in die Spec-Erstellung ein.

Schritt 2: Claude führt ein kurzes Interview: Scope-Grenze, Datenbank, Design-Wünsche. Wenn du keine Idee hast, was die App NICHT tun soll, macht Claude Vorschläge. Du bestätigst oder passt an. Am Ende bekommst du ein fertiges Spec-Dokument zurück. Die Datenbankfrage wird dabei verständlich erklärt. Die Optionen sind:

- **Supabase:** Eine professionelle Datenbank, kostenlos nutzbar. Du siehst deine Daten in einer einfachen Tabellenansicht im Browser. Das Framework ist dafür ausgelegt.
- **Textdatei-Speicherung:** Einfacher einzurichten, aber nur für lokale Tests auf deinem Computer geeignet oder wenn deine App überwiegend statische Daten verwendet, die sich nicht verändern.

Bestätige deine Wahl. Claude trägt sie ins Spec ein.

Schritt 3: Lies das Spec durch. Wenn etwas falsch ist, sage: *"Ändere [was du ändern willst]."*

Gate 1: Prüfe vor Phase 2

- ☐ Das Spec enthält eine User Story ("Als [Nutzer] möchte ich [Aktion], damit [Ergebnis]")
- ☐ Das Spec enthält mindestens 2 Akzeptanzkriterien
- ☐ Das Spec enthält eine Scope-Grenze ("Die App tut NICHT: [...]")
- ☐ Die Datenbankfrage ist beantwortet und von dir bestätigt
- ☐ Die visuelle Präferenz ist festgelegt ("Standarddesign" oder wie von dir beschrieben)

Alle fünf erfüllt → Sage:

Gate 1 ist erfüllt. Speichere das Spec und den Projektstand. Dann weiter zu Phase 2.

Dann weiter zu Phase 2.

Nicht alle erfüllt → zurück zu Schritt 2.

Phase 2: Initialisierung

Ziel: Lauffähiges Projekt im Browser erreichbar, `_workspace/` eingerichtet, Berechtigungen konfiguriert.

Schritt 1: Claude richtet das Projekt ein:

- richtet die Berechtigungskonfiguration ein (damit du nicht bei jeder Aktion von Claude nach Erlaubnis gefragt wirst)
- erstellt das Next.js-Projekt (die "Basis" deiner App, keine Funktionen oder Features)
- verbindet Supabase für die Datenbank, wenn nötig, und fragt dich vorher um Erlaubnis
- legt Ordner und Dateien an
- startet den lokalen Server (die "Vorschau" der App, lokal auf dem Computer)

Schritt 2: Claude gibt dir eine Adresse, die so aussieht: `http://localhost:3000`

Diese Adresse ist nur für dich erreichbar, noch nicht im Internet. Kopiere die Adresse aus Claude und füge sie in die Adressleiste deines Browsers ein.

Wichtig: Claude hält nach der Initialisierung an und wartet auf deine Gate-2-Bestätigung. Es startet Phase 3 nicht von selbst.

Gate 2: Prüfe vor Phase 3

- ☐ Du siehst eine Startseite im Browser unter der localhost-Adresse (kein leeres Fenster, kein Fehler)
- ☐ Claude hat bestätigt, dass `_workspace/` angelegt ist
- ☐ Claude hat bestätigt, dass die Berechtigungskonfiguration eingerichtet ist

Alle drei erfüllt → Sage:

Gate 2 ist erfüllt. Speichere den Projektstand. Dann weiter zu Phase 3.

Dann weiter zu Phase 3.

Nicht erfüllt → Siehe Abschnitt Fehlereskalationspfad, [Stufe 1](#).

Phase 3: Konstruktion

Ziel: Alle Features aus dem Spec implementiert und im Browser testbar. Visueller Stil aus Phase 1 durchgehend angewandt.

Framing: In Phase 2 hast du ein leeres Projekt gestartet. Phase 3 fügt die echten Features hinzu — je nach Umfang direkt alle auf einmal oder eines nach dem anderen. Claude liest dein Spec und schlägt das nächste Feature vor; du bestätigst oder wählst ein anderes. Du musst keine Feature-Liste manuell eintippen.

Anfragen während Phase 3, die über den Spec-Umfang hinausgehen: Füge während dieser Phase keine Features hinzu, die nicht im Spec stehen. Claude weist diese Anfragen ab und merkt sie sich für die Erweiterungsrunde (Gate 3b).

Schritt 1: Wähle das erste Feature (empfohlen: das Kernfeature aus der User Story).

Schritt 2: Claude implementiert das Feature und erklärt dir, wie du es im Browser testen kannst.

Schritt 3: Teste das Feature im Browser. Funktioniert es? Weiter mit dem nächsten Feature. Funktioniert es nicht? → Fehlereskalationspfad, [Stufe 1](#).

Schritt 4: Sage Claude, welches Feature du als nächstes umgesetzt haben möchtest, oder bestätige das Feature, das Claude aus dem Spec vorschlägt. Claude implementiert das Feature.

Wiederhole Schritte 2–4 bis alle Spec-Features implementiert sind.

Regression Wenn ein Feature, das zuvor funktioniert hat, nach der Implementierung eines neuen Features nicht mehr funktioniert: Gehe zu Fehlereskalationspfad [Stufe 1](#).

Kontext-Window und proaktive Session-Pausen: Bei vielen Features oder langen Sessions kann das Kontext-Window von Claude voll werden. Deine Anfragen verbrauchen mehr Ressourcen bei Claude und das Risiko für ungewolltes Verhalten seitens Claude steigt. Claude weist dich dann aktiv darauf hin und empfiehlt einen Session-Neustart. Du kannst auch jederzeit proaktiv neu starten → sage Claude:

Speichere den aktuellen Zwischenstand. Ich mache eine Pause.

und starte dann eine neue Session mit ST-0 (Anhang B).

Warnung: Kontext-Drift-Symptome Wenn Claude anfängt, sich zu widersprechen oder Entscheidungen zu treffen, die dem Spec widersprechen: → Direkt Fehlereskalationspfad [Stufe 3](#).

Gate 3: Prüfe vor Phase 4

- ☐ Alle Features aus dem Spec sind im Browser erreichbar
- ☐ Jedes Feature liefert das in den Akzeptanzkriterien beschriebene Ergebnis

Beide erfüllt → Sage:

Gate 3 ist erfüllt. Speichere den Projektstand.

Gate 3b: Erweiterungsrunde (optional)

Wann: Nach Gate-3-Bestätigung, bevor Phase 4 beginnt.

Nach Gate 3 fragt Claude immer nach Erweiterungswünschen — das ist keine Pflicht-Runde. „Keine Erweiterungen“ genügt; Claude geht dann direkt zu Phase 4 über.

Zweck: Möchtest du nach Abschluss aller Spec-Features noch etwas hinzufügen? Die Erweiterungsrunde hat einen eigenen Bereich in der Projektdokumentation: deine ursprüngliche Spec bleibt der Referenzpunkt, Extension-Features werden separat notiert.

Ablauf:

- Sage nach Gate-3-Bestätigung:

Ich möchte noch [Feature] hinzufügen.

- Falls du keine Erweiterungen möchtest → sage:

Kein Erweiterungswunsch. Weiter zu Phase 4.

- Claude fragt gezielt nach, was es noch wissen muss. Das ist kein Template, sondern ein natürlicher Dialog.
- Claude implementiert das Feature und erklärt dir den Browserstest.
- Teste im Browser. Funktioniert es nicht? → Fehlereskalationspfad [Stufe 1](#).
- Mehrere Erweiterungen möglich: Schritt wiederholen.

Abschluss → Sage:

Gate 3b ist erfüllt. Speichere den Projektstand.

Dann kommt Phase 4.

Hinweis: Du entscheidest über Umfang und Anzahl der Erweiterungen. Claude trägt die technische Ausführungsverantwortung und weist dich hin, wenn das Kontext-Window knapp wird, blockiert aber keine Features.

Phase 4: Validierung

Ziel: Vollständiger Browser-Test aller User-Story-Flows.

Framing: In Phase 3 hast du jedes Feature einzeln getestet. Phase 4 testet den kompletten Ablauf, so wie ein echter Nutzer von Anfang bis Ende durch die App gehen würde.

Schritt 1: Öffne [Validierungs-Checkliste](#) (Anhang B, letzter Abschnitt) und fülle die Platzhalter mit deinen tatsächlichen Akzeptanzkriterien aus. Das machst du auf Papier oder in einer Textdatei. Dieser Schritt ist dein persönlicher Test-Protokollbogen und kein Prompt an Claude.

Schritt 2: Führe den vollständigen User-Story-Flow einmal im Browser durch.

Schritt 3: Hake jedes Akzeptanzkriterium ab oder dokumentiere die Abweichung.

Schritt 4: Fehler gefunden? → Fehlereskalationspfad, [Stufe 1](#).

Gate 4: Prüfe vor Phase 5

- ☐ Alle User-Story-Flows einmal vollständig im Browser durchgespielt
- ☐ Alle Akzeptanzkriterien erfüllt ODER Abweichungen schriftlich dokumentiert
- ☐ Für jede Abweichung: Ist sie ein Flow-Blocker? Falls ja → Gate nicht erfüllt, zurück zu Fehlereskalation

Alle erfüllt → Sage:

Gate 4 ist erfüllt. Speichere den Projektstand. Dann weiter zu Phase 5.

Weiter zu Phase 5.

Phase 5: Deployment

Ziel: Öffentlich erreichbare URL: navigierbar, kernfunktional, auf einem anderen Gerät zugänglich.

Schritt 1: Vercel-Authentifizierung

Claude initiiert den Vercel-Login selbstständig. Es öffnet ein Fenster in deinem Browser. Das ist der Vercel-Login, diesen musst du im Browser erlauben.

Sollte Claude nicht automatisch fortfahren, frage, ob es funktioniert hat.

Falls Claude dich auffordert, den Login-Befehl selbst im Terminal (Befehlszeile/Konsole deines Geräts) auszuführen, sage einfach:

```
mach du es.
```

Claude übernimmt es dann selbst.

Schritt 2: Vercel-Verbindungstest. Sage Claude:

```
Liste meine Vercel-Projekte auf.
```

Claude gibt eine Liste zurück oder informiert dich, dass du noch keine Vercel-Projekte hast.

Wenn eine Fehlermeldung erscheint: richte die initiale Verbindung zwischen Claude und Vercel ein. Du kannst die Einrichtungsanleitung (separates Dokument) dafür verwenden.

Schritt 3 → Sage Claude:

```
Deploye die App auf Vercel.
```

Claude übernimmt alle weiteren Schritte und gibt dir am Ende die öffentliche URL. Das dauert typischerweise 1–3 Minuten. Wenn beim Deployment etwas schief läuft, liest Claude die Deployment-Logs selbstständig und korrigiert. Du erhältst Rückmeldung von Claude, sobald es fertig ist.

Schritt 4: Öffne die URL und prüfe, ob die App geladen wird.

Schritt 5: Führe den User-Story-Flow einmal über die öffentliche URL durch.

Wenn das Deployment länger als 10 Minuten dauert und keine Rückmeldung kommt, sage:

```
Prüfe ob das letzte Vercel-Deployment erfolgreich war oder noch läuft.
```

Gate 5: Prototyp fertiggestellt, wenn:

- ☐ Die App ist unter einer öffentlichen URL erreichbar (nicht localhost)
- ☐ Alle User-Story-Flows funktionieren über die öffentliche URL
- ☐ URL auf einem anderen Gerät oder Browser getestet

Alle drei erfüllt → Sage:

Gate 5 ist erfüllt. Speichere die finale URL und den Projektstand.

Claude schreibt die URL in deinen Projektordner und kennt sie von nun an.

Prototyp fertig.

Deine App ist jetzt öffentlich im Internet erreichbar.

Nach dem Deployment: App weiterentwickeln

Wenn du nach Gate 5 Änderungen oder neue Features hinzufügen möchtest, gehst du so vor:

Schritt 1: Starte eine neue Session mit ST-0:

```
Lies _workspace/Framework.md und _workspace/project_state.md.  
Gib mir eine kurze Zusammenfassung: In welcher Phase sind wir, was ist das letzte  
abgeschlossene Gate, was ist der nächste Schritt? Dann führe mich weiter.
```

Schritt 2: Claude erkennt automatisch, dass Gate 5 bestätigt ist, und fragt dich, was du ändern oder ergänzen möchtest.

Schritt 3: Beschreibe dein Feature oder deine Änderung, wie in Phase 3, ein Feature nach dem anderen.

Schritt 4: Claude implementiert und deployt automatisch neu, oder fragt dich, ob es jetzt deployen soll. Deployment ist nötig, damit Änderungen auf deiner öffentlichen URL sichtbar sind, nicht nur auf deinem Computer.

Diese Änderungen laufen im Post-Deployment-Modus, außerhalb der formalen Gate-Struktur der Phasen 1–5.

Hinweis: Falls Claude dich je auffordert, etwas im Terminal (Befehlszeile/Konsole deines Geräts) einzugeben oder einen Befehl selbst auszuführen, sage einfach:

```
mach du es.
```

Claude führt es dann selbst aus. Das gilt für alle Phasen und den Post-Deployment-Modus.

Anhang A: Fehlereskalationspfad

Wenn etwas nicht funktioniert, gehe die Stufen der Reihe nach durch.

Wichtig: Claude bemerkt technische Fehler (Build-Fehler, Deployment-Fehler) in der Regel selbst und korrigiert sie ohne deinen Eingriff. Der Fehlereskalationspfad ist für Situationen, in denen du im Browser etwas siehst, das nicht stimmt, oder in denen Claude in einer Phase feststeckt.

Fallback-Phrase bei falscher Delegation: Falls Claude dich je auffordert, einen Befehl im Terminal (Befehlszeile/Konsole deines Geräts) einzugeben oder etwas selbst auszuführen, das außerhalb deines Browser-Bereichs liegt, sage einfach:

```
mach du es.
```

Claude führt es dann selbst aus.

Schnellcheck vor Stufe 1: Wenn die Seite nicht lädt, sage zunächst:

```
Starte den Server neu.
```

Claude startet ihn neu. Hilft das nicht, weiter mit Stufe 1.

Stufe 1: Sichtbares Problem beschreiben (immer zuerst)

```
Es gibt ein Problem.
```

```
Was ich sehe: [Beschreibung des sichtbaren Problems im Browser –  
z.B. "Die Seite ist leer", "Der Button reagiert nicht",  
"Das Formular lässt sich nicht absenden"]
```

```
Was ich erwartet hatte: [Soll-Zustand]
```

```
Meine letzte Anweisung war: [Kopie des letzten Prompts; scrolle im Chat nach oben und  
kopiere deinen letzten Satz an Claude]
```

Claude diagnostiziert und behebt selbstständig. Falls nach mehreren Versuchen (~20 Minuten) kein Fortschritt erkennbar ist → [Stufe 2a](#).

Stufe 2a: Letztes Feature rückgängig machen (wenn Stufe 1 nicht hilft)

```
Das zuletzt implementierte Feature hat wahrscheinlich das Problem verursacht.  
Entferne es vollständig und stelle den Stand davor wieder her.  
Speichere danach den aktuellen Stand.
```

Danach: Starte eine neue Session mit ST-0, um das Feature erneut zu versuchen. Claude kennt durch den gespeicherten Stand den aktuellen Fortschritt und kann das Feature neu implementieren. → Gelingt das nicht, weiter mit [Stufe 2b](#).

Stufe 2b: Stand wiederherstellen (wenn Stufe 2a nicht hilft)

Die aktuelle Phase ist in einem inkonsistenten Zustand.
Lies `_workspace/project_state.md` und stelle den Stand nach Gate [Nummer] wieder her.
Beginne Phase [Nummer] neu.

Danach: Gate dieser Phase erneut prüfen.

Stufe 3: Neue Session (bei Kontext-Drift)

Hinweis: Stufe 3 ist unabhängig von Stufe 2. Sie ist keine Folge-Eskalation nach 2b, sondern wird ausschließlich bei Kontext-Drift genutzt.

Sage als erstes:

Speichere den aktuellen Zwischenstand. Ich mache eine Pause.

Starte eine neue Konversation. Sage jetzt:

Lies `_workspace/Framework.md` und `_workspace/project_state.md`.
Das ist eine Session-Neustart-Situation.
Prüfe den aktuellen Dateizustand des Projekts.
Falls Reparaturarbeit nötig ist, führe sie aus.
Falls ein Neuaufbau notwendig ist, sage mir das zuerst; ich bestätige dann.
Fahre danach mit der nächsten offenen Phase fort.

Falls noch kein `_workspace/` -Ordner existiert (allererste Session vor Gate 2):

Starte neu mit Phase 1 und beschreibe deine App-Idee in natürlicher Sprache.

Anhang B: Trigger-Phrasen und Validierungs-Checkliste

ST-0: Session-Start

Lies `_workspace/Framework.md` und `_workspace/project_state.md`.
Gib mir eine kurze Zusammenfassung: In welcher Phase sind wir, was ist das letzte abgeschlossene Gate, was ist der nächste Schritt? Dann führe mich weiter.

Gate-Trigger-Phrasen

Gate 1:

Gate 1 ist erfüllt. Speichere das Spec und den Projektstand. Dann weiter zu Phase 2.

Gate 2:

Gate 2 ist erfüllt. Speichere den Projektstand. Dann weiter zu Phase 3.

Gate 3:

Gate 3 ist erfüllt. Speichere den Projektstand.

Gate 3b:

Gate 3b ist erfüllt. Speichere den Projektstand.

Gate 4:

Gate 4 ist erfüllt. Speichere den Projektstand. Dann weiter zu Phase 5.

Gate 5:

Gate 5 ist erfüllt. Speichere die finale URL und den Projektstand.

Zwischenspeichern:

Speichere den aktuellen Zwischenstand. Ich mache eine Pause.

Validierungs-Checkliste (Phase 4)

Vor Verwendung: Ersetze die Platzhalter mit deinen tatsächlichen Akzeptanzkriterien. Dies ist kein Prompt; fülle es auf Papier oder in einer Textdatei aus.

- Vollständigen User-Story-Flow im Browser durchgespielt
(Schritt für Schritt wie ein echter Nutzer)
 - Akzeptanzkriterium 1: [Text aus Spec]
 - Erfüllt / □ Abweichung: [Notiz]
 - Akzeptanzkriterium 2: [Text aus Spec]
 - Erfüllt / □ Abweichung: [Notiz]
- [weitere Kriterien ergänzen]

Für jede Abweichung:

Ist sie ein Flow-Blocker? Ja → Gate 4 nicht erfüllt, Fehlereskalation Stufe 1
Nein → als akzeptable Abweichung notieren

Anhang C: Kurzglossar

Begriff	Erklärung
Agent / Claude	KI-System, das Aufgaben selbstständig in mehreren Schritten ausführt, ohne dass du jeden Schritt einzeln anweisen musst. Claude liest auch Fehler-Logs und korrigiert technische Fehler selbst.
Commit	Speicherpunkt im Projektcode mit Kommentar. Claude macht Commits automatisch.
Kontext-Window	Das Arbeitsgedächtnis von Claude innerhalb einer Session. Wenn es voll wird, kann Claude frühere Entscheidungen vergessen oder sich widersprechen; das Ergebnis ist Kontext-Drift. Reaktion: Fehlereskalationspfad Stufe 3 (Session-Neustart mit ST-0).
Kontext-Drift	Wenn Claude in einer langen Session anfängt, frühere Entscheidungen zu ignorieren, sich zu widersprechen oder Features neu zu erklären, die bereits funktionieren. Ursache: Kontext-Window voll. Reaktion: Direkt Fehlereskalationspfad Stufe 3 (Session-Neustart mit ST-0).
Deployment	Veröffentlichung der App auf Vercel. Die App ist danach öffentlich im Internet unter einer festen URL erreichbar.
FRAMEWORK.md	Eine Datei, die Claude als Prozessmodell und interne Arbeitsanweisung dient. Du gibst sie Claude in Session 1 (in den Chat ziehen). Claude kopiert sie in Phase 2 nach <code>_workspace/FRAMEWORK.md</code> und liest sie danach am Anfang jeder Session automatisch.
Gate	Binärer Kontrollpunkt zwischen zwei Phasen. Erfüllt oder nicht erfüllt, kein "teilweise". Verhindert, dass Fehler einer Phase in die nächste mitgeschleppt werden.
inputs/	Unterordner in <code>_workspace/</code> . Lege hier Dateien ab, die Claude als Referenz nutzen soll: PDFs, CSVs, Tabellen oder Brand-Guidelines. Teile Claude dann den Dateinamen mit.
localhost	Die Adresse deines Projekts auf deinem eigenen Computer. Beispiel: <code>http://localhost:3000</code> . Nur du siehst diese Seite; sie ist noch nicht im Internet. Nach dem Deployment ersetzt die Vercel-URL diese Adresse.
MCP (Model Context Protocol)	Technische Verbindung, die Claude erlaubt, dein Vercel- und Supabase-Konto direkt zu steuern.
project_state.md	Eine Datei in <code>_workspace/</code> , die Claude nach jedem Gate automatisch schreibt und aktualisiert. Sie enthält dein Spec-Dokument, die abgeschlossenen Gates und den Projektstand. Du musst sie nie öffnen oder bearbeiten.
Scope-Grenze	Die explizite Liste der Dinge, die die App NICHT tun soll. Verhindert, dass Claude Features implementiert, die du nicht beantragt hast.

Begriff	Erklärung
Session	Eine Konversation mit Claude. Eine Session endet wenn du das Fenster schließt oder zu lange wartest. Beim nächsten Start verwendest du ST-0. Claude liest dann FRAMEWORK.md und project_state.md und führt dich weiter.
Spec-Dokument	Schriftliche Anforderungsdefinition: User Story + Akzeptanzkriterien + Scope-Grenze + Datenbankentscheidung + visuelle Präferenz. Das Spec wird in Phase 1 erstellt und von Claude in <code>_workspace/project_state.md</code> gespeichert.
<code>_workspace/</code>	Ein Ordner im Projektverzeichnis, den Claude in Phase 2 anlegt. Er enthält <code>project_state.md</code> , <code>FRAMEWORK.md</code> und <code>inputs/</code> . Dieser Ordner wird nicht deployed und ist ausschließlich für das Framework-Management und deine Referenzmaterialien bestimmt.
